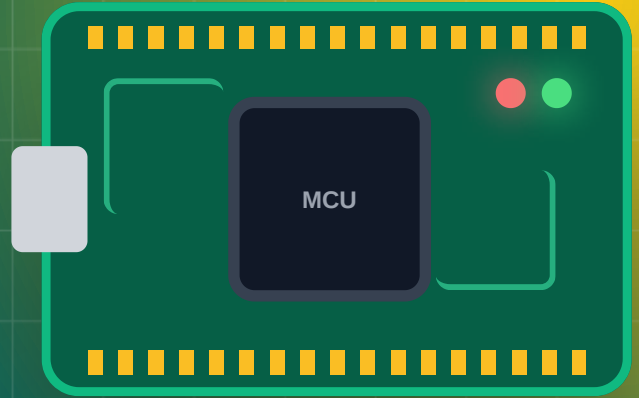




A COMPLETE BEGINNER-TO-PRO LESSON

Code That Touches the Real World

Embedded development — electronics, microcontrollers, embedded C, interrupts, sensors and protocols, RTOS, IoT, embedded Linux, building products, and how to earn from it.

Embedded C

GPIO · PWM

I²C · SPI · UART

FreeRTOS

IoT · MQTT

STM32 · ESP32 · Pico

Prepared for Samuel Nathaniel · September 2026

What you'll learn

0	Start here	2
	What embedded development is, and why it pays	
1	Electronics you actually need	4
	Just enough to build safely and confidently	
2	Inside a microcontroller	6
	A whole computer on one small chip	
3	Embedded C essentials	8
	The language firmware is written in	
4	GPIO, timers and PWM	10
	Making pins do things, on time	
5	Interrupts	12
	Reacting instantly without wasting the CPU	
6	Sensors, ADC and communication protocols	14
	How chips talk to the world and to each other	
7	How firmware is built, flashed and debugged	16
	From C files to a running chip	
8	Real-time operating systems	18
	Doing many things at once, on time	
9	IoT and connectivity	20
	Connecting devices to the internet	
10	Low power, reliability and testing	22
	Devices that run for years without anyone touching them	
11	Embedded Linux	23
	When a microcontroller isn't enough	
12	From prototype to product	24
	Turning a breadboard into something you can sell	
13	Careers and making money	25
	Jobs, freelancing and products	
14	Your 6-month roadmap	27
	From zero to your first real device and first client	
15	Resources, glossary and quiz	29
	Keep going	

0

CHAPTER 0

Start here

What embedded development is, and why it pays

Embedded development is writing the software that lives *inside* things: the washing machine, the prepaid meter, the car's brakes, the insulin pump, the solar inverter, the drone, the smartwatch. There are billions of these devices, far more than laptops and phones, and each one needs firmware that is small, fast, reliable and safe.

Understand

Electronics basics, microcontrollers, memory, interrupts, protocols.

Build

Embedded C, drivers, sensors, RTOS, IoT devices — with tested code.

Earn

Jobs, freelance firmware, and products you can sell in Nigeria and abroad.

Where embedded systems hide

Area	Examples
Energy	Solar charge controllers, inverters, battery management, prepaid meters
Transport	Car engine and brake controllers, GPS trackers, EV chargers
Health	Pulse oximeters, glucose meters, infusion pumps, wearables
Home & industry	Smart locks, fridges, factory PLCs, pumps, CCTV, POS terminals
Agriculture	Soil-moisture sensors, automated irrigation, cold-room monitors

How it differs from what you already do

Orbit Rush (web/app)	Embedded firmware
Gigabytes of RAM, a full operating system	Often tens of kilobytes of RAM, and no operating system at all
Crashes? Reload the page.	Crashes can mean a stuck motor, a dead battery, or a safety hazard
<code>requestAnimationFrame</code> game loop	A <code>for(;;)</code> main loop plus interrupts from hardware
Bugs show in the browser console	Debug with LEDs, a serial port, a debugger probe and an oscilloscope
JavaScript / Java	C (mainly), C++, increasingly Rust; MicroPython for prototypes

◆ Your head start

You already think in **state machines** (home → playing → pause), **fixed time steps** (`tick(dt)`), **tests that must pass**, and **checksums** (your backup codes). All of these are daily tools in firmware.

■ How to use this lesson

- ✓ Read a chapter a day; do each **Try it** box.
- ✓ Code marked **TESTED** was compiled and run; **COMPILES FOR CORTEX-M4** was compiled for a real Arm microcontroller target; **EXAMPLE · NEEDS SDK** shows real APIs but needs the vendor's SDK to build.
- ✓ Start with a **Raspberry Pi Pico** or **ESP32** — cheap, well documented, and easy to buy.
- ✓ Jump to **Chapter 14** for the 6-month plan and shopping list.

1

CHAPTER 1

Electronics you actually need

Just enough to build safely and confidently

Voltage (V)

Electrical “pressure”. MCU pins are usually **3.3 V** (older boards 5 V).

Current (A)

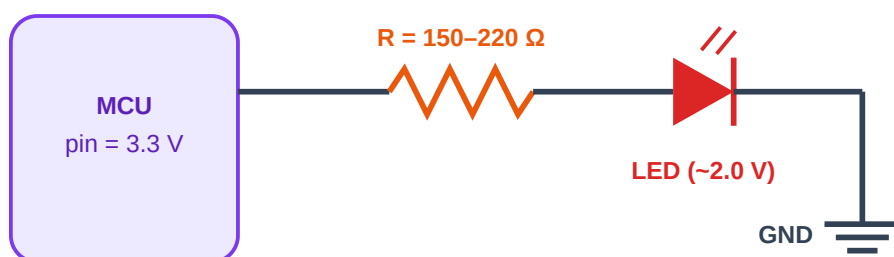
How much charge flows. A pin can safely supply only a few **milliamps**.

Resistance (Ω)

Opposes current. Resistors set currents and divide voltages.

$$V = I \times R \cdot P = V \times I \cdot \text{battery hours} \approx \text{capacity (mAh)} \div \text{average current (mA)}$$

■ Your first circuit: an LED that doesn't burn out



Ohm's law: $V = I \times R$

Resistor voltage
 $= 3.3 - 2.0 = 1.3\text{ V}$

Target current $\approx 8\text{ mA}$

$R = 1.3 \div 0.008 \approx 163\ \Omega$

→ use **150 Ω or 220 Ω**

■ Digital inputs: pull-ups and pull-downs

An input pin connected to nothing is **floating** — it reads random 0s and 1s. A **pull-up** resistor (e.g. 10 k Ω to 3.3 V) holds it at 1 until a button pulls it to ground. Most MCUs have **internal** pull-ups you can enable in software.

■ Driving bigger things

- **Transistors / MOSFETs** let a tiny pin current switch a big load (motors, LED strips, buzzers).
- **Relays** switch mains-powered equipment — always with a driver transistor and a **flyback diode**.
- **Motor drivers** (H-bridges) control direction and speed of DC motors.
- **Decoupling capacitors** (100 nF next to every chip's power pin) keep supply noise from resetting your MCU.

■ 3.3 V vs 5 V

Connecting a 5 V sensor output to a 3.3 V-only pin can damage it. Check the datasheet for “5 V tolerant”, or use a **level shifter** or a resistor divider.

! Safety first

- **Never** work on mains (220–240 V AC) wiring until you are trained — use ready-made, certified relay modules and keep mains in a closed box.
- Lithium batteries can catch fire if shorted or over-charged: use protected cells and proper charger chips.
- Don't power motors directly from an MCU pin or its 3.3 V regulator.

▶ Try it

Use a free online simulator such as **Wokwi** (wokwi.com) to wire an LED, resistor and button to a virtual Pico, ESP32 or Arduino — no hardware needed yet.

2

CHAPTER 2

Inside a microcontroller

A whole computer on one small chip

A **microcontroller (MCU)** puts a CPU, program memory (flash), working memory (RAM) and hardware helpers called **peripherals** onto one chip that costs from a few hundred naira to a few thousand.



Popular families

Family	Core	Why people use it
AVR (Arduino Uno)	8-bit AVR	Simple, huge community, great for first steps
STM32 (STMicroelectronics)	Arm Cortex-M0 to M7, M33	Industry workhorse; hundreds of variants
RP2040 / RP2350 (Raspberry Pi Pico)	Cortex-M0+ / M33, plus RISC-V on RP2350	Cheap, superb docs, clever programmable I/O
ESP32 (Espressif)	Xtensa or RISC-V	Built-in Wi-Fi and Bluetooth — IoT favourite
nRF52 / nRF54 (Nordic)	Cortex-M4 / M33	Bluetooth Low Energy, wearables
PIC, MSP430, NXP, Renesas...	Various	Common in industrial, automotive and low-power products

■ The memory map: hardware at addresses

Core peripherals (NVIC, SysTick)	0xE000 0000
Peripherals (GPIO, UART, ...)	0x4000 0000
SRAM (variables, stack)	0x2000 0000
Code: flash (program, constants, vector table)	0x0000 0000

Every Arm Cortex-M chip follows this standard layout. Writing to address 0x4002 0014 does not touch memory — it changes the voltage on real pins.

(On STM32, flash is mirrored at 0x0800 0000 — the datasheet tells you the exact map for every chip.)

■ Datasheet and reference manual — your two best friends

Datasheet

Pins, voltages, currents, speeds, package drawings. “Can this chip do it, and how do I wire it?”

Reference manual

Every peripheral and every register bit, often 1,000+ pages. “How do I make it do it?” You search it; nobody reads it cover to cover.

★ The real skill

Being able to open an unfamiliar chip’s manual, find the right register, and make a peripheral work is what separates embedded engineers from hobbyists. Practise it deliberately.

3

CHAPTER 3

Embedded C essentials

The language firmware is written in

C gives you direct control of memory and hardware with almost no overhead. Firmware C is ordinary C plus a handful of habits.

1. Exact-width types

Use `uint8_t`, `int16_t`, `uint32_t` from `<stdint.h>`. On a microcontroller, “`int`” might be 16 or 32 bits — registers need exact sizes.

2. Registers as structs, with `volatile`

Real code from an STM32F4 layout: the GPIO port’s registers become a struct placed at the port’s base address. `volatile` forces the compiler to perform every read and write, because the hardware can change the value (or care about every write).

```

gpio_regs.c COMPILES FOR CORTEX-M4 C

#include <stdint.h>

/* A register map as a struct: offsets must match the datasheet exactly. */
typedef struct {
    volatile uint32_t MODER; /* 0x00: pin mode (input/output/...) */
    volatile uint32_t OTYPER; /* 0x04: push-pull / open-drain */
    volatile uint32_t OSPEEDR; /* 0x08: output speed */
    volatile uint32_t PUPDR; /* 0x0C: pull-up / pull-down */
    volatile uint32_t IDR; /* 0x10: input data (read pins) */
    volatile uint32_t ODR; /* 0x14: output data (write pins) */
    volatile uint32_t BSRR; /* 0x18: atomic set/reset */
} gpio_t;

_Static_assert(__builtin_offsetof(gpio_t, BSRR) == 0x18, "layout must match datasheet");

#define GPIOA ((gpio_t *)0x40020000u) /* from the datasheet */
#define RCC_AHB1ENR (*(volatile uint32_t *)0x40023830u) /* clock enables */
#define LED 5u /* LED on pin PA5 */

void led_init(void) {
    RCC_AHB1ENR |= (1u << 0); /* switch on GPIOA's clock first */
    GPIOA->MODER &= ~(3u << (LED * 2)); /* clear the pin's 2 mode bits */
    GPIOA->MODER |= (1u << (LED * 2)); /* 01 = general-purpose output */
}

void led_on(void) { GPIOA->BSRR = 1u << LED; } /* set bit */
void led_off(void) { GPIOA->BSRR = 1u << (LED + 16); } /* reset bit */

```

3. The bit-manipulation toolkit

bit tricks

C

```
reg |= (1u << n);           /* set bit n */
reg &= ~(1u << n);          /* clear bit n */
reg ^= (1u << n);           /* toggle bit n */
if (reg & (1u << n)) { }    /* test bit n */
reg = (reg & ~(3u << 10)) | (2u << 10); /* write a 2-bit field at bits 10-11 */
```

4. Integer (fixed-point) maths

Many MCUs have no floating-point unit, so firmware often scales integers instead of using `float`:

adc.h TESTED

C

```
#include <stdint.h>

/* 12-bit ADC (0..4095) with a 3.3 V reference -> millivolts, integers only */
static inline uint32_t adc_to_mv(uint16_t raw) {
    return ((uint32_t)raw * 3300u + 2047u) / 4095u; /* rounded */
}

/* Battery measured through a divider: 100k on top, 33k below.
   Vbat = Vpin * (100 + 33) / 33 */
static inline uint32_t battery_mv(uint16_t raw) {
    return adc_to_mv(raw) * 133u / 33u;
}
```

Host test run

```
raw 0 -> 0 mV, raw 2048 -> 1650 mV, raw 4095 -> 3300 mV
battery: raw 3000 -> 9745 mV
```

5. Habits that keep firmware alive for years

Do

Allocate memory statically; use `const` for tables (they stay in flash); check every return value; keep functions small; turn on all warnings (`-Wall -Wextra`).

Avoid

`malloc` in long-running code (fragmentation); recursion (stack overflow); undefined behaviour (overflowing signed ints, reading uninitialised variables); giant functions in interrupts.

i C++ and Rust

Modern C++ (without exceptions and heavy libraries) is common in firmware, and **Rust** is growing fast for safety. Learn C first — every vendor's SDK, datasheet example and job interview assumes it.

4

CHAPTER 4

GPIO, timers and PWM

Making pins do things, on time

■ GPIO: general-purpose input/output

Each pin can be an **input** (read a button or sensor), an **output** (drive an LED or transistor), or handed to a peripheral (UART, PWM, ADC). Outputs can be **push-pull** (drives high and low) or **open-drain** (only pulls low; used by I²C).

```
blink.c — Raspberry Pi Pico SDK EXAMPLE · NEEDS SDK

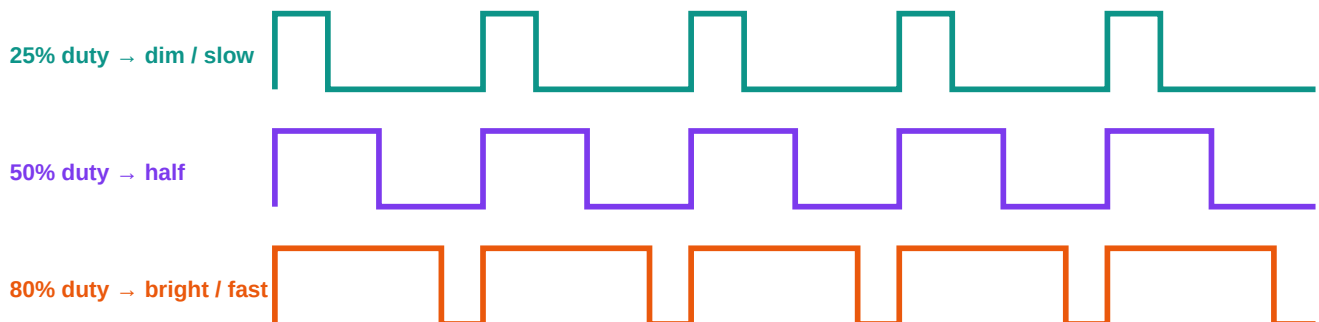
#include "pico/stdlib.h"

int main(void) {
    const unsigned led = PICO_DEFAULT_LED_PIN;
    gpio_init(led);
    gpio_set_dir(led, GPIO_OUT);
    while (true) {
        gpio_put(led, 1);
        sleep_ms(250);
        gpio_put(led, 0);
        sleep_ms(250);
    }
}
```

■ Timers: stop using delay loops

A **timer** is a hardware counter clocked from the system clock. It can interrupt you every 1 ms (a “tick”), measure pulse widths, or generate waveforms — all while the CPU does other work. Replace `sleep_ms()` in real products with a timer tick and a state machine, just like Orbit Rush’s `tick(dt)`.

■ PWM: pretending to be analog



Pulse-width modulation switches a pin on and off very fast. The fraction of time it is on (the **duty cycle**) controls LED brightness, motor speed, servo angle and buzzer tones.

■ Debouncing a button

Mechanical buttons “bounce” — one press produces several on/off flickers within a few milliseconds. This small state machine, called every 1 ms from a timer, reports exactly one press:

```
● ● ● debounce.h TESTED C

#include <stdbool.h>
#include <stdint.h>

/* Call every 1 ms with the raw pin level. Returns true once per real press. */
typedef struct { uint8_t count; bool stable; } debounce_t;

static inline bool debounce_update(debounce_t *d, bool raw_pressed) {
    if (raw_pressed == d->stable) { d->count = 0; return false; }
    if (++d->count < 20) return false;    /* must hold for 20 ms */
    d->stable = raw_pressed;
    d->count = 0;
    return d->stable;                    /* report press, not release */
}
```

Host test: bouncy input, a glitch, then a second real press

debounce: all tests passed (2 presses detected)

▶ Try it

On a Pico or Arduino: make an LED breathe smoothly using PWM, then add a debounced button that cycles through speeds.

5

CHAPTER 5

Interrupts

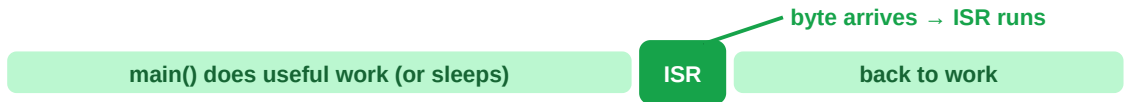
Reacting instantly without wasting the CPU

POLLING



"Is there a byte yet? No. No. No. No. ..." — the CPU is busy doing nothing, and can still miss events.

INTERRUPT



The hardware taps the CPU on the shoulder only when something happens.

An **interrupt** is a hardware signal that pauses your program, runs a small function called an **interrupt service routine (ISR)**, and then returns exactly where it left off. On Arm Cortex-M, the **NVIC** (nested vectored interrupt controller) manages priorities so urgent interrupts can pre-empt less urgent ones.

Receiving serial data with an interrupt and a ring buffer

The ISR stores each byte in a **ring buffer**; `main()` reads them when it is ready. Only the ISR writes `head` and only `main()` writes `tail`, so no locking is needed for one producer and one consumer.

```

● ● ● ringbuf.h TESTED C

#include <stdbool.h>
#include <stdint.h>

/* A ring buffer (FIFO): the interrupt writes bytes in, main() reads them out. */
#define RB_SIZE 64u /* must be a power of two */

typedef struct {
    volatile uint8_t data[RB_SIZE];
    volatile uint16_t head; /* next slot to write */
    volatile uint16_t tail; /* next slot to read */
} ringbuf_t;

static inline bool rb_put(ringbuf_t *rb, uint8_t byte) {
    uint16_t next = (rb->head + 1u) & (RB_SIZE - 1u);
    if (next == rb->tail) return false; /* full: drop the byte */
    rb->data[rb->head] = byte;
    rb->head = next;
    return true;
}

static inline bool rb_get(ringbuf_t *rb, uint8_t *byte) {
    if (rb->tail == rb->head) return false; /* empty */
    *byte = rb->data[rb->tail];
    rb->tail = (rb->tail + 1u) & (RB_SIZE - 1u);
    return true;
}

```

Host test run

ring buffer: all tests passed

```
uart_isr.c COMPILES FOR CORTEX-M4 C

#include "ringbuf.h"

#define UART1_SR (*(volatile uint32_t *)0x40011000u) /* status */
#define UART1_DR (*(volatile uint32_t *)0x40011004u) /* data */
#define RXNE      (1u << 5) /* "byte received" flag */

static ringbuf_t rx; /* shared by the ISR and main() */

/* Runs automatically when a byte arrives. Keep it SHORT. */
void USART1_IRQHandler(void) {
    if (UART1_SR & RXNE)
        rb_put(&rx, (uint8_t)UART1_DR); /* reading DR clears RXNE */
}

/* Called from main(): returns -1 when nothing is waiting. */
int uart_read_byte(void) {
    uint8_t b;
    return rb_get(&rx, &b) ? b : -1;
}
```

Register addresses follow the STM32F4's USART1. To actually receive interrupts you also enable the peripheral's RX interrupt bit and the USART1 line in the NVIC.

■ The golden rules of ISRs

- ✓ **Keep them short** — copy data, set a flag, get out. Do heavy work in `main()` or a task.
- ✓ **Share variables safely** — mark them `volatile`; for multi-byte data, briefly disable interrupts (a **critical section**) or use atomic operations.
- ✓ **Never block** — no `printf`, no delays, no waiting for other hardware inside an ISR.
- ✓ **Clear the interrupt flag**, or the ISR will fire again forever.

★ The watchdog timer

A **watchdog** resets the chip if your code stops “feeding” it — e.g. if it gets stuck in a loop. Every product that must run unattended (meters, pumps, trackers) should use one.

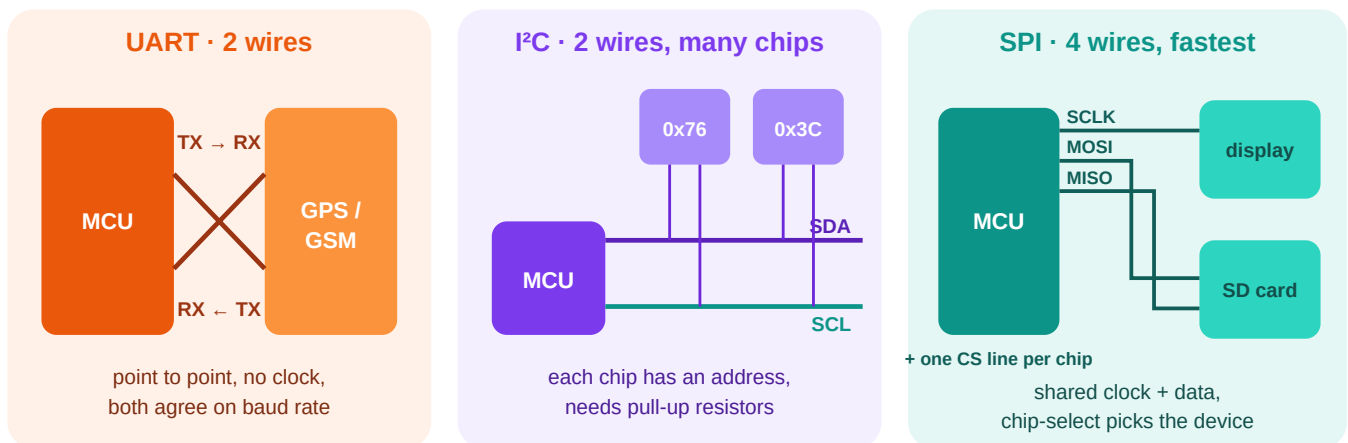
Sensors, ADC and communication protocols

How chips talk to the world and to each other

■ Analog to digital: the ADC

An **ADC** turns a voltage into a number. A 12-bit ADC gives 0–4095 across its reference voltage (often 3.3 V). To measure something bigger — like a 12 V battery — use a **voltage divider** (the `battery_mv()` function in Chapter 3 does exactly that). Average several readings to reduce noise.

■ The three buses you will use every week



	UART	I ² C	SPI
Wires	TX, RX (+GND)	SDA, SCL (+GND)	SCLK, MOSI, MISO, CS per device
Typical speed	9,600 – 1,000,000 baud	100 kHz, 400 kHz, 1 MHz	1 – 50+ MHz
Devices	2 (point to point)	Many, by 7-bit address	Many, one CS line each
Typical parts	GPS, GSM modems, debug console	Temperature/pressure sensors, small OLEDs, RTC clocks	Displays, SD cards, flash memory, fast ADCs

■ More protocols worth knowing

CAN

Robust bus for cars, trucks and machines.

RS-485 + Modbus

Long cables in factories, solar inverters and energy meters.

USB

Device classes like serial (CDC), keyboards (HID), storage.

1-Wire

Cheap temperature sensors like the DS18B20.

I²S

Digital audio: microphones and amplifiers.

Ethernet

Wired networking for gateways and industrial devices.

▶ Try it

Buy a BME280 (temperature, humidity, pressure) breakout. Wire it over I²C, scan the bus to find its address, then read the chip ID register and print it over the serial port. Reading one register correctly from a datasheet is a huge milestone.

How firmware is built, flashed and debugged

From C files to a running chip

The build pipeline



- **Cross-compiler** — runs on your PC, produces code for the MCU (e.g. `arm-none-eabi-gcc`, or `clang`).
- **Linker script** — tells the linker where flash and RAM are, and where to put code, constants and variables.
- **Build system** — Make or CMake; vendor IDEs (STM32CubeIDE, ESP-IDF, Arduino IDE) or PlatformIO in VS Code.

What happens at power-on



The first words of flash are the **vector table**: the initial stack pointer and the address of every interrupt handler. Startup code copies initialised variables (`.data`) from flash to RAM and zeroes the rest (`.bss`) before calling `main()`.

Your debugging toolkit

Tool	What it shows	Starter option
Serial log	printf-style messages over UART/USB	Any USB-to-serial adapter; built in on most boards
Debug probe (SWD/JTAG)	Breakpoints, step through code, inspect registers and memory	ST-Link (on Nucleo boards), Raspberry Pi Debug Probe, J-Link
GDB + OpenOCD	The open-source debugger stack behind most IDEs	Free
Multimeter	Voltage, current, continuity	Any decent digital meter
Logic analyser	Decodes UART/I ² C/SPI signals on real wires	Cheap 8-channel USB analyser + free PulseView software
Oscilloscope	Real analog waveforms, noise, timing	Entry-level digital scope when you are ready

★ Debugging mindset

Change one thing at a time. Check power first, then clocks, then pins, then code. When a bus doesn't work, look at the actual wires with a logic analyser — the problem is visible more often than you expect.

8

CHAPTER 8

Real-time operating systems

Doing many things at once, on time

When a device must read sensors, update a display, talk to the network and watch buttons all at once, a single main loop gets messy. A **real-time operating system (RTOS)** lets you split the work into **tasks** and guarantees that urgent tasks run on time.

Tasks

Independent functions, each with its own stack and priority.

Scheduler

Always runs the highest-priority task that is ready.

Queues

Pass data safely between tasks and from ISRs.

Mutexes

Protect a shared resource (e.g. the I²C bus).

Semaphores

Signal “an event happened” between tasks.

Timers

Run callbacks after a delay or periodically.

```
two FreeRTOS tasks and a queue EXAMPLE · NEEDS SDK C

#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"

static QueueHandle_t readings;

static void sensor_task(void *arg) {
    for (;;) {
        int32_t mv = read_battery_mv();           /* your driver */
        xQueueSend(readings, &mv, 0);
        vTaskDelay(pdMS_TO_TICKS(1000));         /* once per second */
    }
}

static void display_task(void *arg) {
    int32_t mv;
    for (;;) {
        if (xQueueReceive(readings, &mv, portMAX_DELAY) == pdTRUE)
            show_voltage(mv);                     /* your display code */
    }
}

int main(void) {
    readings = xQueueCreate(8, sizeof(int32_t));
    xTaskCreate(sensor_task, "sensor", 256, NULL, 2, NULL);
    xTaskCreate(display_task, "display", 256, NULL, 1, NULL);
    vTaskStartScheduler();                       /* never returns */
}
```

■ Which RTOS?

RTOS	Notes
FreeRTOS	The most widely used; built into ESP-IDF; small and simple. Start here.
Zephyr	Linux Foundation project; many boards, drivers, networking and Bluetooth built in. Growing fast in industry.
ThreadX, embOS, RT-Thread, NuttX	Also common in commercial and consumer products.

! Classic RTOS bugs

- **Stack overflow** — each task's stack is fixed; enable stack checking.
- **Priority inversion** — a low-priority task holds a mutex a high-priority task needs; use mutexes with priority inheritance.
- **Deadlock** — two tasks each waiting for the other's lock; always take locks in the same order.

9

CHAPTER 9

IoT and connectivity

Connecting devices to the internet

Sensor
temp, current...MCU
reads + filtersRadio
Wi-Fi · LoRa · GSMMQTT broker
cloud or localDatabase
+ dashboardPhone
alerts, control

■ Choosing a radio

Technology	Range	Power	Good for
Wi-Fi	Building	High	Mains-powered devices near a router (ESP32)
Bluetooth LE	~10-50 m	Very low	Wearables, phone-controlled gadgets
LoRa / LoRaWAN	Kilometres	Very low	Farms, water tanks, remote meters — small data, long battery life
Cellular: 2G/4G, LTE-M, NB-IoT	Anywhere with coverage	Medium-high	Vehicle trackers, remote sites, prepaid meters (check local network support)
Zigbee / Thread / Matter	Home mesh	Low	Smart-home devices

■ Protocols on top

- **MQTT** — lightweight publish/subscribe messaging; the standard for IoT. Devices publish to topics like `farm/tank1/level`.
- **HTTP/REST** and **WebSockets** — simple when bandwidth and power allow.
- **OTA updates** — update firmware over the air. Essential for real products; always keep a fallback image.

■ A MicroPython taste (ESP32)

```
publish a reading over MQTT EXAMPLE · MICROPYTHON PY

import network, time
from umqtt.simple import MQTTClient
from machine import ADC, Pin

wlan = network.WLAN(network.STA_IF)
wlan.active(True)
wlan.connect("MyWiFi", "password")
while not wlan.isconnected():
    time.sleep(0.5)

adc = ADC(Pin(34))
client = MQTTClient("tank-1", "broker.example.com")
client.connect()
while True:
    client.publish(b"farm/tank1/raw", str(adc.read()))
    time.sleep(60)
```

! Security is not optional

Never ship default passwords. Use TLS for network traffic, sign your firmware updates, lock the debug port on production units, and store keys securely. Insecure IoT devices get hijacked into botnets.

◆ Your web skills pay off here

An IoT product is half firmware, half dashboard. You can already build the web or Android app that shows the data — that makes you unusually complete as an IoT developer.

10

CHAPTER 10

Low power, reliability and testing

Devices that run for years without anyone touching them

■ Designing for battery life

Most battery devices spend 99% of their time **asleep**. Wake up, measure, send, sleep again.

$$\text{Average current} = (\text{active mA} \times \text{active time} + \text{sleep mA} \times \text{sleep time}) \div \text{total time}$$

Example: sensor waking every 10 minutes	Value
Awake: 2 seconds at 40 mA (radio on)	80 mA·s per cycle
Asleep: 598 seconds at 0.01 mA	≈ 6 mA·s per cycle
Average current	$(80 + 6) \div 600 \approx \mathbf{0.14 \text{ mA}}$
2,000 mAh battery	$2,000 \div 0.14 \approx 14,000 \text{ h} \approx \mathbf{1.6 \text{ years}}$ (before real-world losses)

The same device that never sleeps (40 mA constantly) would last about **2 days**. Sleep modes, turning off unused peripherals and sending less often matter enormously.

■ Reliability checklist

- ✓ **Watchdog** enabled and fed only when the system is truly healthy.
- ✓ **Brown-out detection** — reset cleanly when the supply dips (very common with unstable power).
- ✓ **Store settings safely** — flash wears out; use wear levelling and checksums (like your Orbit Rush backup codes!).
- ✓ **Handle every error** — sensors unplugged, network down, corrupted packets.
- ✓ **Log a reset reason** at boot so field failures can be diagnosed.

■ Testing firmware

Unit tests on your PC

Keep logic (parsers, filters, state machines, ring buffers) free of hardware so it compiles on your laptop — exactly how the ring buffer and debounce code in this lesson were tested. Frameworks: Unity, CppUTest, GoogleTest.

On the hardware

Hardware-in-the-loop (HIL) rigs drive real devices automatically. Also: long soak tests, power-cycling tests, and temperature tests.

i Safety-critical industries

Cars, aircraft and medical devices follow standards such as ISO 26262, DO-178C and IEC 62304, and coding rules like MISRA C. Engineers who know them are well paid.

11

CHAPTER 11

Embedded Linux

When a microcontroller isn't enough

Cameras, routers, POS terminals, car dashboards and smart TVs need more memory, a screen, networking and files. They run **Linux** on a **microprocessor (MPU)** or single-board computer instead of bare firmware on an MCU.

	Microcontroller	Embedded Linux
Example	Pico, STM32, ESP32	Raspberry Pi, BeagleBone, i.MX, Rockchip boards
RAM	KB – a few MB	Hundreds of MB – GB
Boot time	Milliseconds	Seconds
Real-time	Hard real-time	Soft (better with PREEMPT_RT)
Power	Microamps asleep	Watts
Software	Your code, maybe an RTOS	Full OS, Python, databases, web servers

Skills in this area

- **Linux command line, shell scripting, systemd.**
- **Cross-compiling** and building custom images with **Buildroot** or the **Yocto Project**.
- **Device tree** — describes the board's hardware to the kernel.
- **Kernel drivers** in C, and user-space access via `/dev`, `/sys`, I²C and SPI device files.
- **Bootloaders** (U-Boot) and secure, reliable updates (A/B partitions).

★ A common architecture

Many products combine both: a Linux processor for the UI and networking, plus a small MCU for real-time control and low-power standby. Knowing both sides makes you very employable.

12

CHAPTER 12

From prototype to product

Turning a breadboard into something you can sell

Schematic
KiCad

Layout
place + route

Gerbers
fab files

Fab + assembly
JLCPCB, PCBWay...

Bring-up
test, fix

Enclosure
3D print / mould

■ Designing your own circuit board

- **KiCad** is free, professional-grade PCB software. Draw the schematic, lay out the board, export Gerber files.
- Low-cost PCB services (e.g. JLCPCB, PCBWay) make and even assemble small batches and ship worldwide — check current prices, shipping times and import duties.
- Start with a simple two-layer board: an ESP32 module, a regulator, a sensor connector and a few LEDs.

■ What a real product needs

Power design

Regulators, protection against reverse polarity and surges.

Enclosure

3D-printed for prototypes; injection-moulded at volume. Water and dust ratings (IP65...).

Manufacturing test

A quick automated test for every unit before it ships.

Certification

Radio and safety approvals (e.g. FCC, CE; in Nigeria, radio equipment may need NCC type approval).

Bill of materials

Cost every part; choose components that are easy to buy and not near end-of-life.

Support

OTA updates, documentation, warranty and repairs.

◆ Start small, sell early

You don't need a factory. Many successful hardware businesses began by hand-assembling 10–50 units for real customers, learning from their feedback, then improving the design.

Roles

Role	What you do	Key skills
Firmware engineer	Write and debug MCU code and drivers	C, peripherals, RTOS, debugging
Embedded Linux engineer	Board bring-up, drivers, Yocto images	Linux, C, device tree, networking
IoT developer	Connected devices end to end	ESP32/Nordic, MQTT, cloud, apps
Test / validation engineer	HIL rigs, automated tests	Python, lab equipment, C
Automotive / medical / aerospace	Safety-critical firmware	C, standards (ISO 26262, IEC 62304)
Hardware product founder	Design, build and sell your own device	All of the above + business

Four ways to earn

1. Remote jobs and contracts

Firmware and IoT work is often remote-friendly. A strong GitHub portfolio with real boards and videos opens doors worldwide.

2. Freelance projects

Businesses and start-ups need prototypes, driver fixes, IoT dashboards and firmware updates. Your firmware + web + Android mix is rare and valuable.

3. Local products and services

Build and sell devices that solve local problems (next page), or install and maintain monitoring systems for businesses.

4. Teaching and content

Kits, workshops for students, YouTube tutorials — once you've built enough to teach.

Product ideas with real demand in Nigeria

Problem	Device idea	Core parts
Unstable grid, expensive fuel	Solar + battery monitor that shows charge, load and alerts on phone	ESP32, current/voltage sensors, app
Generator running costs	Generator run-hour and fuel-level tracker for businesses	MCU, fuel sensor, GSM or Wi-Fi
Food and vaccine spoilage	Cold-room / freezer temperature logger with SMS alerts	Temperature sensors, GSM module, battery backup

Water supply	Tank level monitor and automatic pump controller	Ultrasonic sensor, relay, ESP32
Farming	Soil-moisture and irrigation controller	Moisture sensors, LoRa, solar power
Vehicle security, logistics	GPS tracker with engine immobiliser	GPS + GSM/LTE module, MCU

! Be realistic

Income depends on skill, portfolio, reliability and how well you find clients — not on the topic alone. Expect your first paid work to come after several finished projects. Avoid anyone selling “guaranteed job” courses.

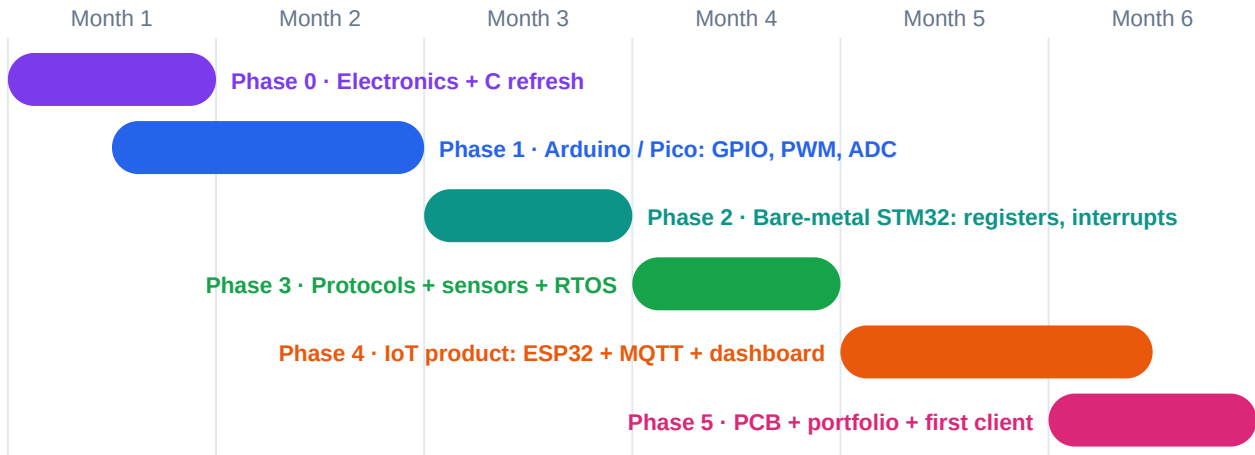
14

CHAPTER 14

Your 6-month roadmap

From zero to your first real device and first client

About **1-2 hours a day**. Every project goes on GitHub with photos, a wiring diagram and a short video.



Phase 0 · Month 1 · Electronics + C refresh

free

- Chapters 1-3 of this lesson; Ohm's law exercises; wire circuits in **Wokwi**.
- C practice on your laptop: pointers, structs, bit operations; write the ring buffer yourself and test it.

Phase 1 · Months 1-2 · First board

≈ US\$5-25

- Raspberry Pi Pico (C SDK) or Arduino Uno. Blink → button with debounce → PWM breathing LED → read a potentiometer with the ADC → print over serial.

Phase 2 · Months 2-3 · Bare metal

≈ US\$15-25

- An STM32 Nucleo board (built-in ST-Link debugger). Blink using only registers, then a timer interrupt, then a UART driver with a ring buffer. Step through it in the debugger.

Phase 3 · Months 3-4 · Sensors, protocols, RTOS

≈ US\$15-30

- I²C temperature sensor + SPI or I²C display; a cheap logic analyser to see the signals. Port the project to FreeRTOS with two tasks and a queue.

Phase 4 · Months 4-5 · A complete IoT product

≈ US\$10-30

- ESP32 device that sends readings over MQTT to a dashboard you build (your web/Android skills). Add deep sleep, a watchdog and OTA updates. Pick one idea from Chapter 13.

Phase 5 · Months 5-6 · PCB, portfolio, first client

- Design a simple KiCad board for your IoT device and order a few. Write up 3-4 projects. Offer to build one monitoring device for a local business, or apply for remote firmware roles.

■ Starter shopping list

Item	Why
Breadboard, jumper wires, resistors, LEDs, buttons, potentiometer	Every first experiment
Raspberry Pi Pico (or Arduino Uno) + ESP32 dev board	Learning board + Wi-Fi board
STM32 Nucleo board	Industry-style bare-metal learning with a built-in debugger
Sensors: BME280, DS18B20, ultrasonic HC-SR04, small OLED display	Practise I ² C, 1-Wire, timing and SPI/I ² C displays
Digital multimeter + USB logic analyser	See what is really happening
Soldering iron, solder, flux	Headers, modules, your own PCBs

Buy from reputable local electronics sellers or online stores; prices and availability vary, so compare before buying and order parts with long shipping times early.

■ Your first week

Day	Task
1	Read Chapters 0-1. Create a free Wokwi account; blink a virtual LED on a Pico.
2	In Wokwi, add a button with an internal pull-up; toggle the LED on each press.
3	Read Chapter 3. Compile and test <code>ringbuf.h</code> on your laptop with gcc.
4	Add the debounce code to your Wokwi project; count presses on the serial monitor.
5	Make an LED breathe with PWM.
6	Order your first board and starter parts.
7	Push everything to GitHub with a README; answer the quiz in Chapter 15.

15

CHAPTER 15

Resources, glossary and quiz

Keep going

■ Books

Book	Best for
<i>Making Embedded Systems</i> — Elecia White	The best overall introduction to thinking like a firmware engineer
<i>Programming Embedded Systems</i> — Barr & Massa	Classic, practical C on hardware
<i>The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors</i> — Joseph Yiu	How Cortex-M really works
<i>Test-Driven Development for Embedded C</i> — James Grenning	Testing firmware properly
<i>Practical Electronics for Inventors</i> — Scherz & Monk	Electronics reference
<i>Mastering Embedded Linux Programming</i> — Frank Vasquez & Chris Simmonds	Embedded Linux

■ Free online

Practice and docs

Wokwi simulator · Raspberry Pi Pico C SDK docs · ESP-IDF docs · STM32 reference manuals · FreeRTOS and Zephyr docs

Learning

Interrupt (Memfault's embedded blog) · **Embedded Artistry** · **Phil's Lab** (PCB design, YouTube) · **DigiKey** and **Adafruit** tutorials · **Embedded.fm** podcast

■ Glossary

ADC — converts voltage to a number.

BSP — board support package: low-level code for a board.

Bootloader — small program that loads or updates firmware.

Brown-out — supply voltage dipping too low.

Critical section — code run with interrupts disabled.

DMA — hardware that moves data without the CPU.

Duty cycle — on-time fraction of a PWM signal.

Firmware — software stored in a device's flash.

Flash — non-volatile program memory.

GPIO — general-purpose input/output pin.

HAL — hardware abstraction layer (vendor library).

I²C — 2-wire addressed bus.

ISR — interrupt service routine.

JTAG / SWD — debug interfaces.

Linker script — memory layout for the linker.

MCU — microcontroller.

MQTT — lightweight IoT messaging.

NVIC — Cortex-M interrupt controller.

OTA — over-the-air update.

Peripheral — hardware block inside the MCU.

Pull-up — resistor holding a line high.

PWM — pulse-width modulation.

Register — memory-mapped hardware control word.

RTOS — real-time operating system.

SPI — fast 4-wire bus.

UART — asynchronous serial port.

Watchdog — timer that resets a stuck system.

■ Quick quiz

1. An LED drops 2.0 V on a 3.3 V pin. What resistor gives about 10 mA?
2. Why must hardware registers be declared `volatile`?
3. What is the difference between polling and interrupts?
4. Name two rules for writing an ISR.
5. Which bus uses an address for each device on two wires?
6. What does a 50% PWM duty cycle mean?
7. A 12-bit ADC reads 2048 with a 3.3 V reference. About what voltage is that?
8. What does a watchdog timer do?
9. Name two tasks the startup code performs before `main()`.
10. Why does sleeping between measurements matter so much for battery life?

Answers: 1 $(3.3 - 2.0) \div 0.010 = 130 \Omega \rightarrow$ use 150Ω · 2 the hardware can change or react to values, so every access must really happen · 3 polling keeps asking; interrupts let hardware notify the CPU · 4 keep it short, don't block, clear the flag, share data safely · 5 I²C · 6 the pin is on half the time · 7 about 1.65 V · 8 resets the chip if software stops responding · 9 copy `.data` to RAM, zero `.bss`, set up clocks · 10 average current falls from tens of mA to a fraction of a mA, stretching days into years.

■ Build something real.

Every embedded engineer started with a blinking LED. Your advantage is that you can already build the app and dashboard around a device. Finish one useful device for a real person, and you are no longer a beginner — you are a builder.